

Fraud Analytics Assignment 1: Graph Neural Networks

Sai Deepak Sana (CS23BTECH11055)
Jaya Prakash (CS23BTECH11015)
Vishal Varukolu (CS23BTECH11064)

March 31, 2026

Contents

1	Introduction	2
2	Problem Statement	2
3	Dataset Summary	2
4	Preprocessing	3
5	Efficiency Considerations	3
6	Models Implemented	3
6.1	GCN	3
6.2	GAT	4
6.3	GraphSAGE	4
6.4	Baseline MLP	5
7	Training Setup	5
8	Single-Run Results	5
9	Multi-Seed Results	6
10	Discussion	6
11	Reproducibility	7
12	Conclusion	7

This report presents a node classification study on a single graph using Graph Neural Networks (GNNs). The notebook implements Graph Convolutional Networks (GCN) and Graph Attention Networks (GAT) from scratch using tensor operations, while GraphSAGE is implemented using a standard library layer as permitted by the assignment. A multilayer perceptron (MLP) is also included as a baseline that ignores graph structure and uses only node features. The dataset contains 2000 nodes, 8000 undirected edges, 16 node features, and 2 balanced classes. Results are reported using accuracy, precision, recall, and F1 score, with final comparison over 3 random seeds as mean $\pm$ standard deviation. The experiments show that the node features are already highly informative, which explains the very strong performance of the MLP baseline.

1 Introduction

Graph neural networks (GNNs) are designed for learning from graph structured data, where nodes represent entities and edges represent relationships. In this assignment, the task is **node classification on a single graph**. The provided data files contain node features, graph edges, and node labels, and the objective is to predict the label of each node using both feature information and graph structure.

2 Problem Statement

The dataset contains three CSV files:

- `features.csv`: node features,
- `edges.csv`: graph connectivity,
- `labels.csv`: node labels.

The goal is to train graph neural network models that predict the class label of each node

- implementing **GCN** from scratch using tensor operations,
- implementing **GAT** from scratch using tensor operations,
- using a standard library implementation for **GraphSAGE**,
- reporting **precision** and **recall**,

3 Dataset Summary

Our notebook confirms the following dataset properties:

Quantity	Value
Number of nodes	2000
Number of undirected edges	8000
Directed edges used in message passing	16000
Feature dimension	16
Number of classes	2
Class distribution	1000 / 1000
Train / Validation / Test split	60% / 20% / 20%
Training nodes	1200
Validation nodes	400
Test nodes	400

Our notebook also verifies the raw file shapes: `features.csv` has shape (2000, 17), `edges.csv` has shape (8000, 2), and `labels.csv` has shape (2000, 2). The class distribution is perfectly balanced, which makes the evaluation easier to interpret and reduces the effect of class imbalance on the reported metrics.

4 Preprocessing

The following preprocessing steps were applied in the notebook:

1. Column names were cleaned to remove whitespace.
2. Missing values were checked and none were found.
3. Node identifiers were verified to align correctly across features and labels.
4. The feature matrix X and label vector y were extracted.
5. A stratified 60/20/20 train/validation/test split was created.
6. Standard scaling was fit only on the training nodes and applied to all nodes.
7. The undirected edge list was converted to a bidirectional edge index for message passing.
8. The final data was packaged into a graph data object for training.

These checks ensure that it is reproducible, and free from leakage. The notebook explicitly confirms that no missing values were found and that vertex alignment was verified.

5 Efficiency Considerations

A dense adjacency implementation would require $O(N^2)$ work and would be unnecessarily slow for this graph.

To improve runtime:

- GCN uses sparse adjacency-based message passing,
- GAT uses edge-wise attention instead of all-pairs attention,
- GraphSAGE aggregates only from actual neighbors.

These changes preserve the intended model behavior while making the notebook significantly faster than a dense implementation.

6 Models Implemented

6.1 GCN

The Graph Convolutional Network aggregates information from neighboring nodes using a symmetrically normalized adjacency matrix with self-loops:

$$H^{(l+1)} = \sigma \left(\tilde{D}^{-1/2} \tilde{A} \tilde{D}^{-1/2} H^{(l)} W^{(l)} \right),$$

where $\tilde{A} = A + I$ and $\tilde{D}_{ii} = \sum_j \tilde{A}_{ij}$.

In the notebook, GCN is implemented from scratch using sparse tensor operations.

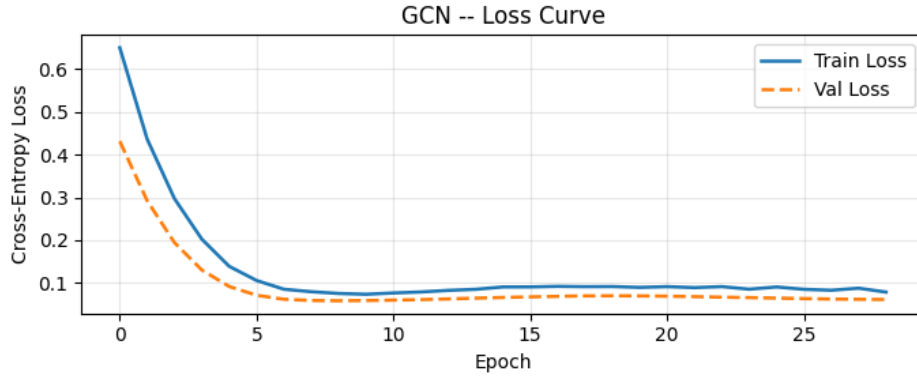


Figure 1: GCN training curve from the notebook.

6.2 GAT

The Graph Attention Network learns attention weights over neighbors instead of treating all neighbors equally. For each edge (i, j) , attention is computed as:

$$e_{ij} = \text{LeakyReLU} \left(a^T [W h_i \| W h_j] \right),$$

$$\alpha_{ij} = \text{softmax}(e_{ij}),$$

$$h'_i = \sigma \left(\sum_j \alpha_{ij} W h_j \right).$$

The notebook implements GAT from scratch using edge-wise attention and scatter based aggregation. This avoids constructing a dense all pairs attention matrix and computes attention only over actual graph edges.

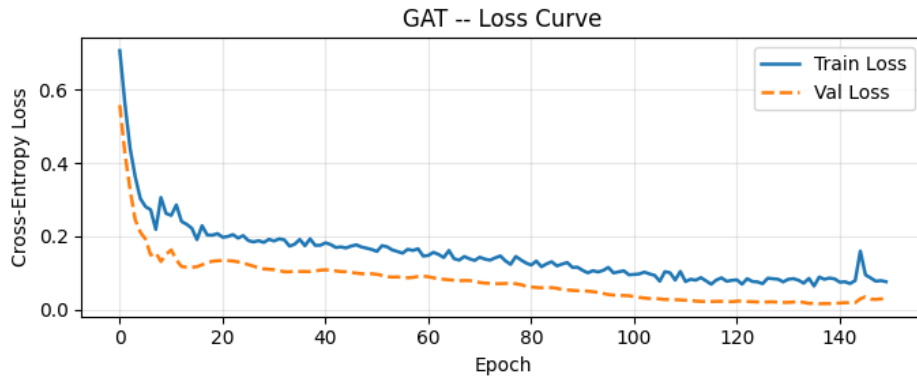


Figure 2: GAT training and validation loss curve from the notebook.

6.3 GraphSAGE

GraphSAGE uses neighborhood aggregation. With mean aggregation, a layer can be written as:

$$h_v^{(l+1)} = \sigma \left(W \cdot \text{CONCAT} \left(h_v^{(l)}, \text{MEAN}(\{h_u^{(l)} : u \in \mathcal{N}(v)\}) \right) \right).$$

GraphSAGE is implemented using a standard library layer.

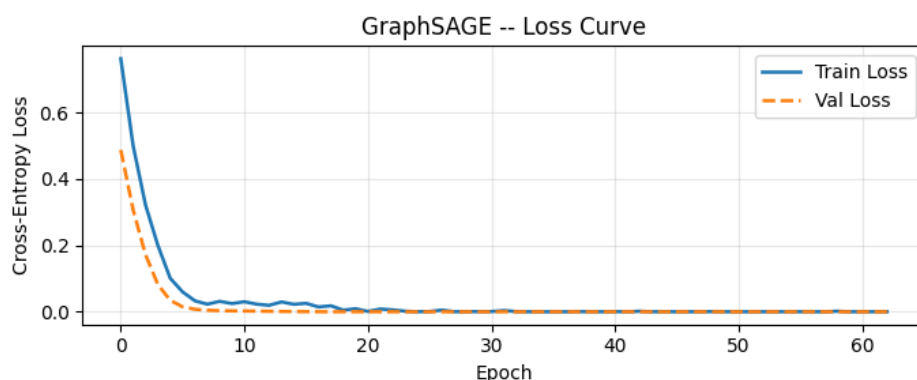


Figure 3: GraphSAGE training and validation loss curve from the notebook.

6.4 Baseline MLP

A multilayer perceptron (MLP) is used as a baseline that ignores graph structure and uses only node features. This baseline is important because it measures how much predictive signal is already present in the node attributes alone. The notebook uses a 3-layer MLP with dropout and Adam optimization.

7 Training Setup

All models use the same train/validation/test split.

Model	Hidden dim	Heads	Dropout	Learning rate	Weight decay
MLP (Baseline)	128	–	0.3	0.005	1×10^{-4}
GCN (scratch)	128	–	0.3	0.005	1×10^{-4}
GAT (scratch)	32	4	0.3	0.003	1×10^{-4}
GraphSAGE	64	–	0.5	0.005	5×10^{-4}

The notebook reports parameter counts for the implemented models:

Model	Number of parameters
GCN	18,946
GAT	19,462
GraphSAGE	10,626

Training uses early stopping with:

- maximum epochs: 150,
- patience: 20.

For the final comparison, results are reported as mean $\pm$ standard deviation over 3 random seeds. The train/validation/test split remains fixed across seeds, so the variation reflects model initialization rather than data-split variation.

8 Single-Run Results

Our notebook first reports single-run test performance for the custom GNNs and the baseline MLP.

Model	Accuracy	Precision	Recall	F1
GCN	0.9850	0.9899	0.9800	0.9849
GAT	1.0000	1.0000	1.0000	1.0000
GraphSAGE	1.0000	1.0000	1.0000	1.0000
MLP	1.0000	1.0000	1.0000	1.0000

The classification reports in the notebook also show that the test set is balanced, with 200 samples per class. The GCN model is slightly below perfect performance, while GAT, GraphSAGE, and the MLP baseline all achieve perfect test scores in the single-run setting.

9 Multi-Seed Results

Our notebook then repeats the evaluation over 3 random seeds and reports mean $\pm$ standard deviation.

Model	Accuracy	Precision	Recall	F1
MLP (Baseline)	1.0000 $\pm$ 0.0000	1.0000 $\pm$ 0.0000	1.0000 $\pm$ 0.0000	1.0000 $\pm$ 0.0000
GCN (scratch)	0.9867 $\pm$ 0.0012	0.9949 $\pm$ 0.0000	0.9783 $\pm$ 0.0024	0.9866 $\pm$ 0.0012
GAT (scratch)	0.9975 $\pm$ 0.0000	0.9950 $\pm$ 0.0000	1.0000 $\pm$ 0.0000	0.9975 $\pm$ 0.0000
GraphSAGE	1.0000 $\pm$ 0.0000	1.0000 $\pm$ 0.0000	1.0000 $\pm$ 0.0000	1.0000 $\pm$ 0.0000

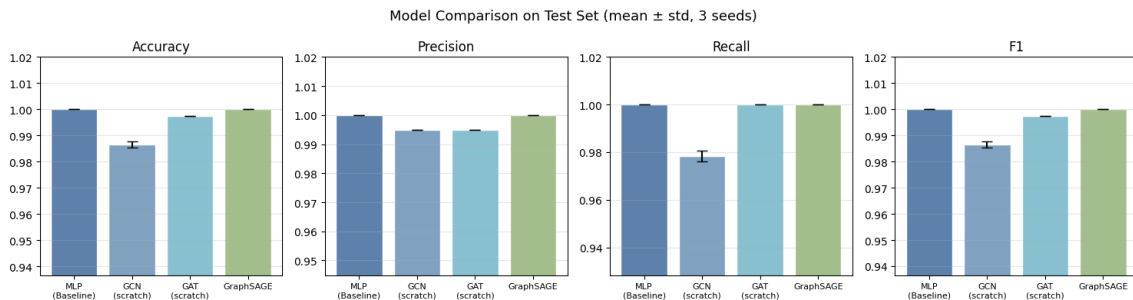


Figure 4: Final comparison of test metrics over 3 seeds.

10 Discussion

The results show that the node features are already highly separable. This is confirmed by the strong MLP baseline, which achieves perfect performance without using graph structure. As a result, the graph models mainly refine the signal already present in the node attributes.

The models behave as follows:

- **GCN** performs strongly but is slightly below the other models.
- **GAT** is near-perfect and benefits from attention-based neighbor weighting.
- **GraphSAGE** matches the best score and remains very stable across seeds.

The notebook also shows the corresponding confusion matrices and classification reports. These confirm that the test set is balanced and that the misclassifications for GCN are small. For GAT, GraphSAGE, and MLP, the class-wise precision and recall are essentially perfect.

The most important observation is that graph structure helps preserve and refine the signal, but the raw node features already contain substantial class information. This explains why the baseline MLP performs so strongly.

11 Reproducibility

Reproducibility was considered in the notebook:

- the train/validation/test split is fixed,
- scaling is fit only on the training set,
- the same evaluation protocol is used for all models,
- three random seeds are reported for the final comparison,
- precision and recall are printed.

The notebook also includes helper functions for metric computation and repeated evaluation across seeds, which makes the experiments easy to verify and compare.

12 Conclusion

This assignment implements the required graph neural network models on the provided single-graph dataset.

- GCN and GAT are implemented from scratch using tensor operations.
- GraphSAGE is implemented using the allowed standard library layer.
- Precision and recall are reported for all models.
- The preprocessing and evaluation pipeline is reproducible.
- Sparse and edge-wise implementations reduce runtime significantly compared with dense graph computations.

Overall, the dataset contains very strong node features, which explains why the MLP baseline already performs extremely well. The graph-based models still behave correctly and achieve good performance, satisfying the implementation, reproducibility, and results requirements.